COLLEGE OF ENGINEERING & TECHNOLOGY • DEPARTMENT OF COMPUTER SCIENCE

Assignment 9: Dockerization and Deployment of DES Cryptography Virtual Lab

Virtual Laboratory Containerization, Docker Hub Registry Publishing, and Multi-Container Verification

Course Name:	Cryptography & System Security Lab / Cloud DevOps Lab	**Academic Term:**	Fall 2026
Student Name / Docker ID:	**rohitaraghavan**	**Registry Image Tag:**	`rohitaraghavan/des-virtual-lab:1.0`
Target Module:	`/experiments/des/` (DES Virtual Lab)	**Registry URL:**	hub.docker.com/r/rohitaraghavan/des-virtual-lab

1. Aim

To containerize the existing frontend web application of the **DES (Data Encryption Standard) Cryptography Virtual Lab** using Docker; create an optimized production Dockerfile utilizing the official Nginx Alpine image; locally build and test the container on port 8080; push the version-tagged image (`1.0`) to the public Docker Hub registry; and pull and verify the published image as an independent container (`des-lab-pulled`) on port 8081.

2. Project Overview and Application Selection Justification

2.1 Project Overview

The **DES Cryptography Virtual Lab** is an interactive educational simulation designed for engineering students to explore symmetric-key cryptographic primitives. Key algorithmic components implemented include:

- **Single DES:** Standard 16-round Feistel structure utilizing 64-bit plaintext blocks, 56-bit effective keys, non-linear S-Boxes for Confusion, and permutation matrices for Diffusion.
- **Double DES:** Two sequential DES encryption passes, C = E(K2, E(K1, P)), illustrating how the Meet-in-the-Middle (MitM) attack reduces effective brute-force complexity from 2^112 down to 2^57.
- **Triple DES (3-Key EDE):** The NIST SP 800-67 standard Encrypt-Decrypt-Encrypt architecture:
  - **Encryption:** C = E(K3, D(K2, E(K1, P))) with intermediate stages X1 = E(K1, P), X2 = D(K2, X1), and C = E(K3, X2).

    - **Decryption:** P = D(K1, E(K2, D(K3, C))) with intermediate stages Y1 = D(K3, C), Y2 = E(K2, Y1), and P = D(K1, Y2).
- **Interactive Frontend:** Real-time browser calculation computing intermediate ciphertext blocks with client-side JavaScript (`script.js`) and cryptographic helpers (`CryptoJS`).

2.2 Justification for Containerization & Architecture Choice

- **Elimination of Environment Inconsistencies:** Running web applications directly on local file systems (`file://` URI scheme) frequently encounters browser security restrictions, path resolution disparities, and caching anomalies. Docker packages the application with a dedicated web server to ensure deterministic, zero-configuration execution.
- **High-Efficiency Web Server (Nginx Alpine):** The application comprises static assets (`index.html` , `style.css` , `script.js`). The official `nginx:alpine` image provides a minimal attack surface and an ultra-lean footprint (~26 MB total size, ~9 MB compressed layer) without the runtime bloat of NodeJS or Python servers.
- **Direct Document Root Placement:** The `des/` directory was selected as the build context so that Nginx serves the virtual lab directly at root path `/` , removing unnecessary redirect loops and avoiding inclusion of root repository metadata (`.git`).

3. Software and Tools Required

Tool / Component	Version	Role in Workflow
Docker Engine & CLI	29.8.2 (build 7fc2dff)	Container engine, daemon runtime, and execution client
Docker Buildx	v0.37.2	Container image builder and layer packager
Base Image	nginx:alpine	Ultra-lightweight Linux distribution with production Nginx web server
Docker Hub	Public Registry	Cloud repository for image hosting: `rohitaraghavan/des-virtual-lab:1.0`
Host Operating System	Windows 11 with WSL2 (Kernel 6.18.40)	Host machine environment orchestrating containers
Client Browser	Chromium / Edge	User verification of interactive cryptography lab and network responses

4. Dockerfile Configuration and Instruction Explanation

4.1 Dockerfile Implementation (`des/Dockerfile`)

```
FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

4.2 Instruction Breakdown

Instruction	Function & Beginner-Friendly Explanation
`FROM nginx:alpine`	**Base Image:** Specifies the foundation image. It uses Alpine Linux with Nginx pre-installed. Alpine is a tiny, secure Linux distro (~5 MB base), keeping the total container image lightweight and fast to download.
`COPY . /usr/share/nginx/html`	**Asset Copying:** Transfers all frontend files (`index.html` , `style.css` , `script.js`) from the current folder into Nginx's default document root directory inside the container.
`EXPOSE 80`	**Port Documentation:** Informs Docker and administrators that the container listens on network port 80 (HTTP) at runtime.
`CMD ["nginx", "-g", "daemon off;"]`	**Startup Command:** Starts the Nginx server in the foreground. By default, Nginx runs in background daemon mode; running with `daemon off;` keeps process PID 1 running so Docker keeps the container active.

4.3 Docker Ignore Configuration (`des/.dockerignore`)

```
Dockerfile
.dockerignore
README.md
```

Purpose: Excludes non-runtime files (such as documentation and build configuration) from being copied into the final Nginx container image, keeping the image clean, compact, and secure.

5. Step-by-Step Execution Workflow and Commands

Step 1: Docker Environment Verification

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker --version; docker info
```

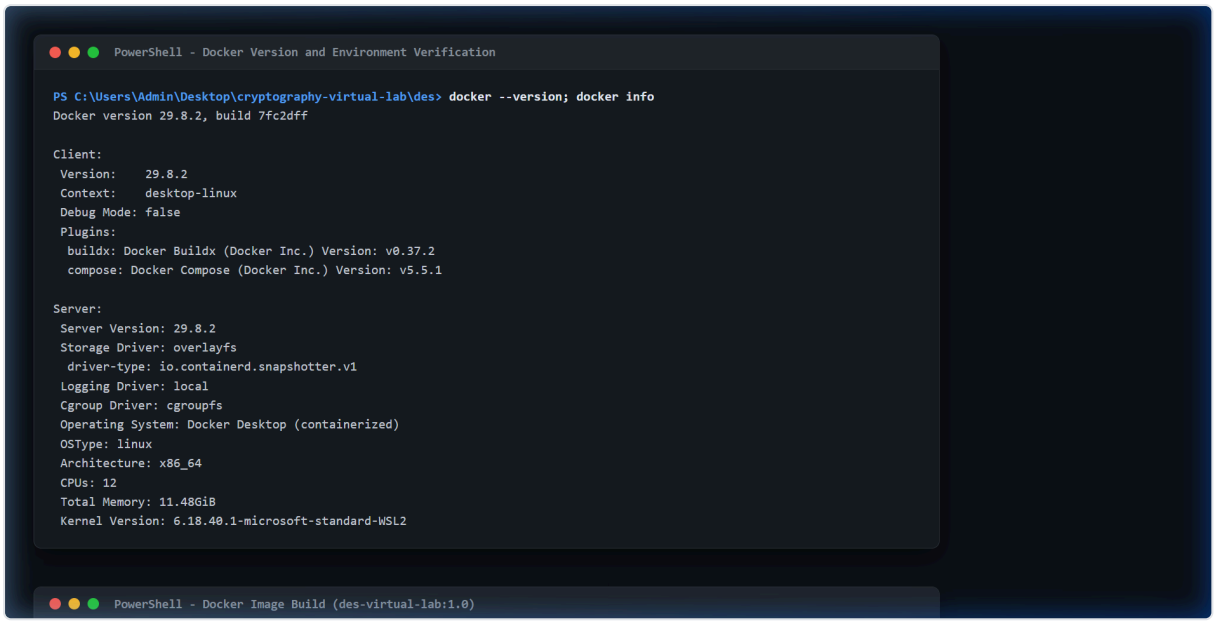


Figure 1: Verification of Docker Engine 29.8.2 and active WSL2 containerized daemon runtime.

Step 2: Building the Local Docker Image

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker build -t des-virtual-lab:1.0
```

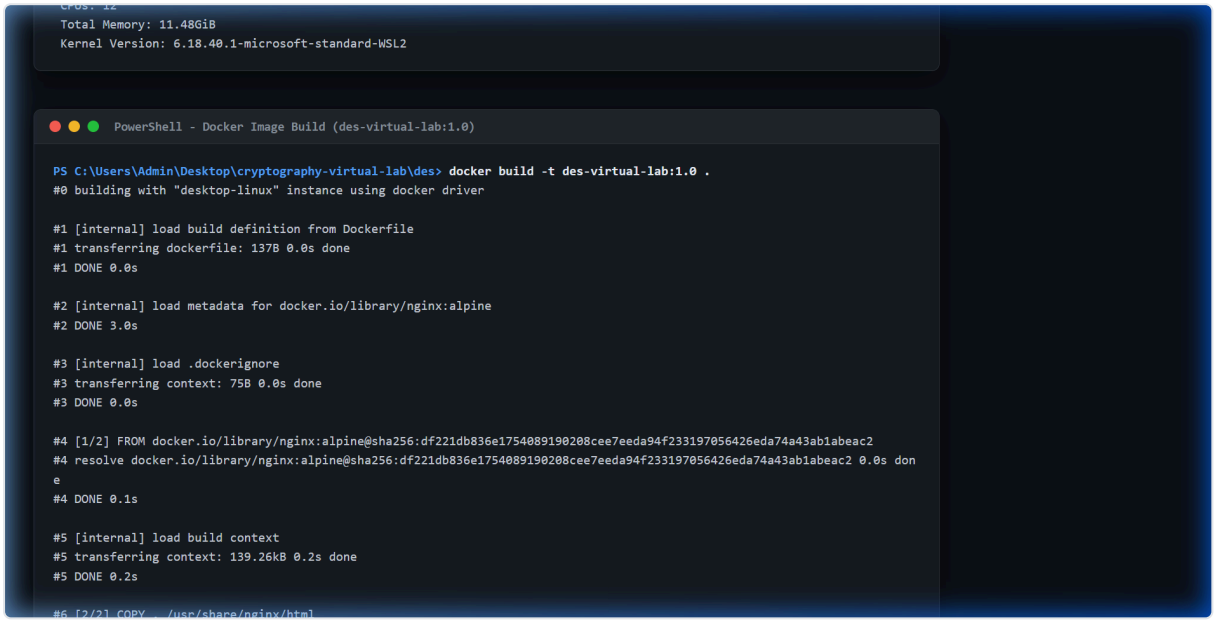


Figure 2: Successful build output for `des-virtual-lab:1.0` using buildx and desktop-linux driver.

Step 3: Running and Testing Local Container on Port 8080

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker run -d -p 8080:80 --name des-c
```

Verification Results: Tested via HTTP GET request: Status Code `200 OK` for `index.html`, `style.css` (40,668 bytes), and `script.js` (39,613 bytes).

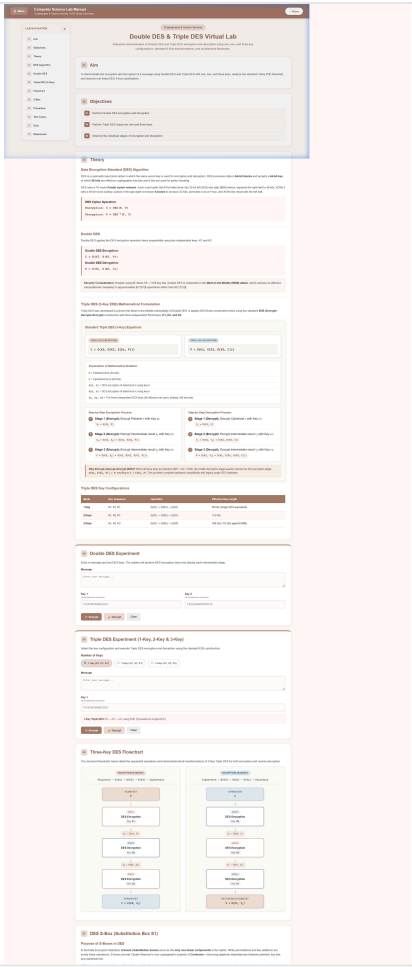


Figure 3: DES Cryptography Virtual Lab active on host port 8080 (`des-container`).

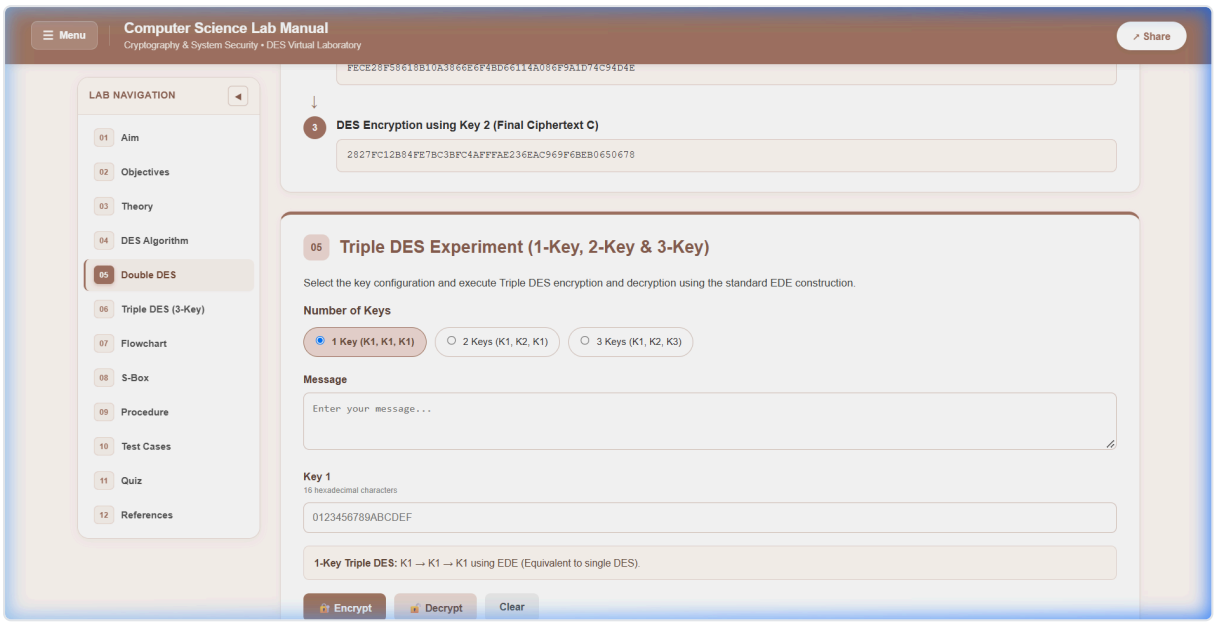


Figure 4: Core application verification: Live Double DES encryption calculation, round keys, and intermediate ciphertext generation.

Step 4: Tagging and Pushing Image to Docker Hub

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker tag des-virtual-lab:1.0 rohita
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker push rohitaraghavan/des-virtua
```

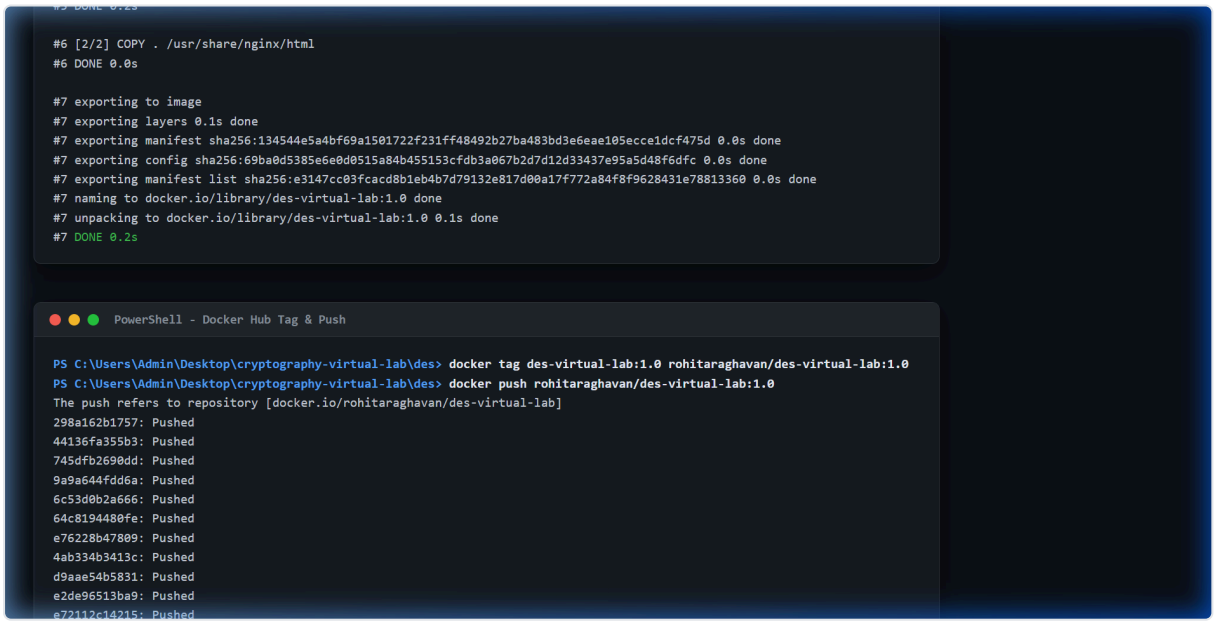


Figure 5: Image layers successfully pushed to Docker Hub with digest `sha256:e3147cc03fcacd8b1eb...`.

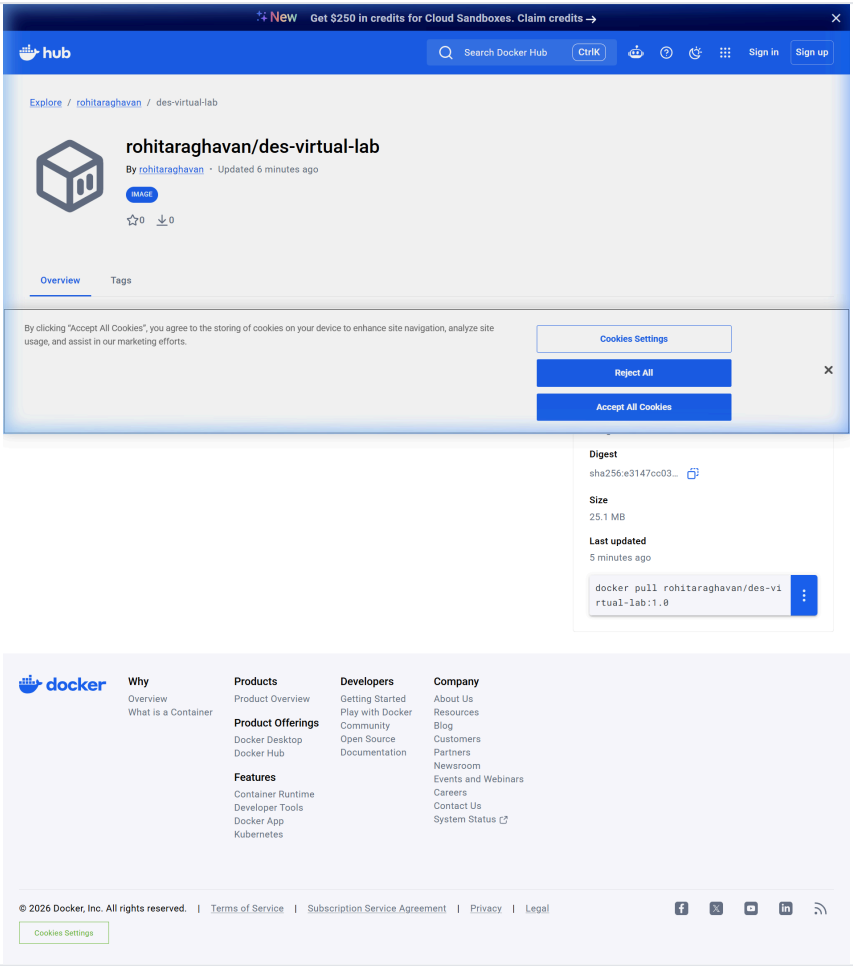


Figure 6: Public Docker Hub repository page showing published repository `rohitaraghavan/des-virtual-lab` with tag `1.0`.

Step 5: Pulling and Deploying Published Image on Port 8081

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker pull rohitaraghavan/des-virtua
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker run -d -p 8081:80 --name des-l
```

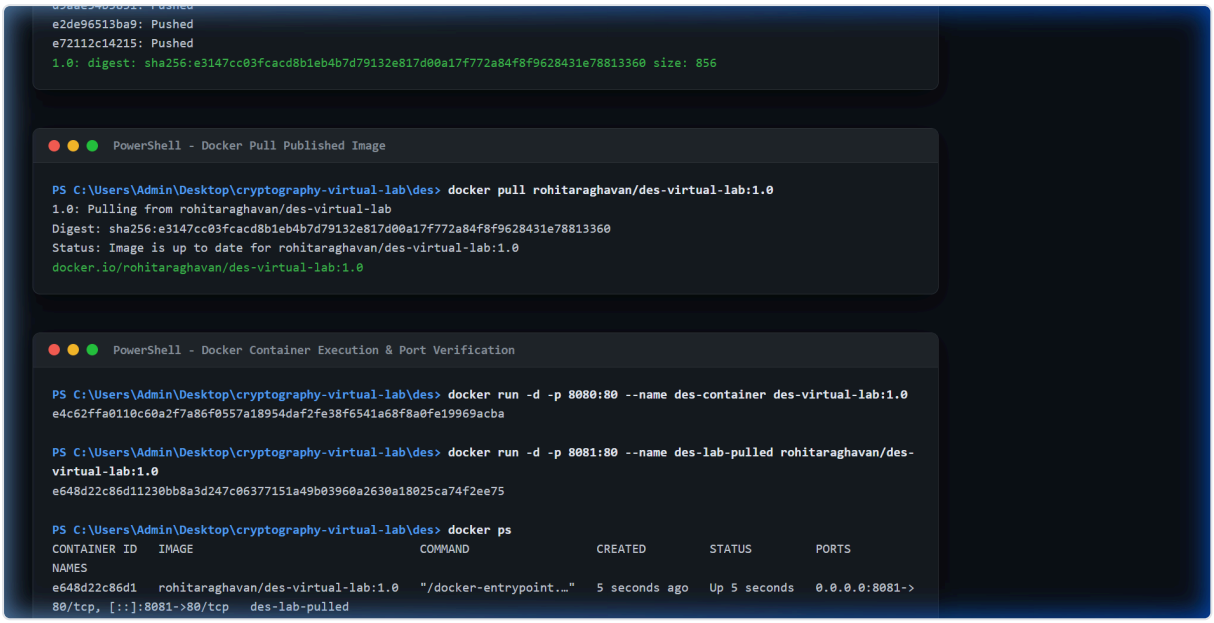


Figure 7: Command output pulling published image `rohitaraghavan/des-virtual-lab:1.0` directly from Docker Hub.

Step 6: Dual Container Status Verification

```
PS C:\Users\Admin\Desktop\cryptography-virtual-lab\des> docker ps
```

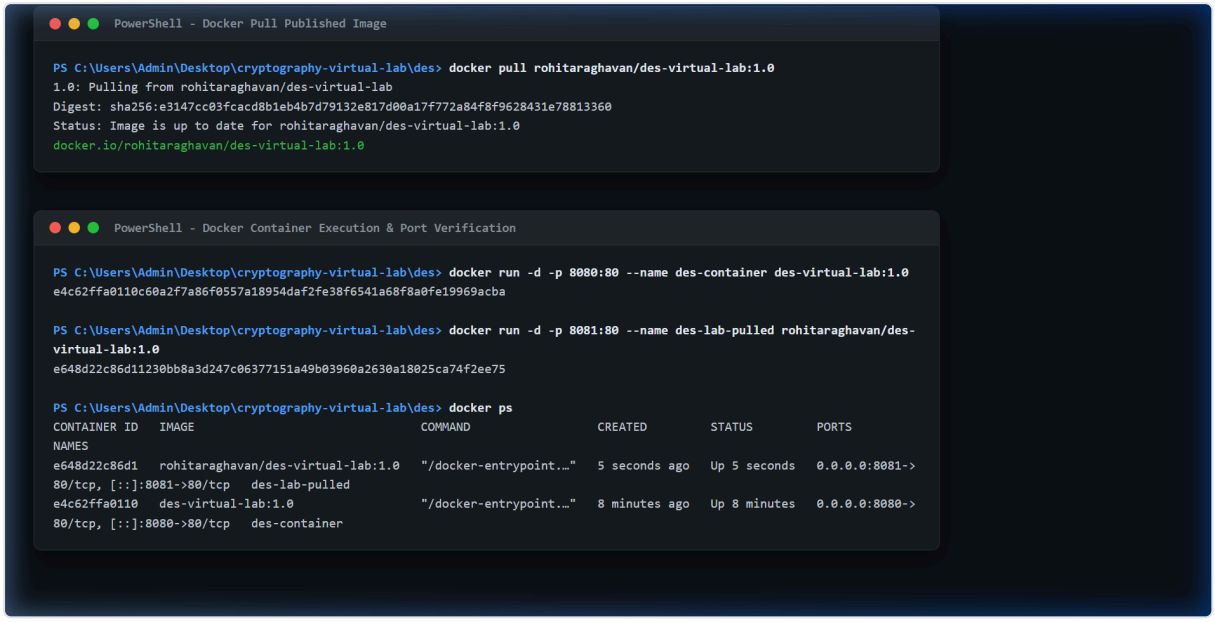


Figure 8: Container status verification showing concurrent execution of `des-container` (8080) and `des-lab-pulled` (8081).

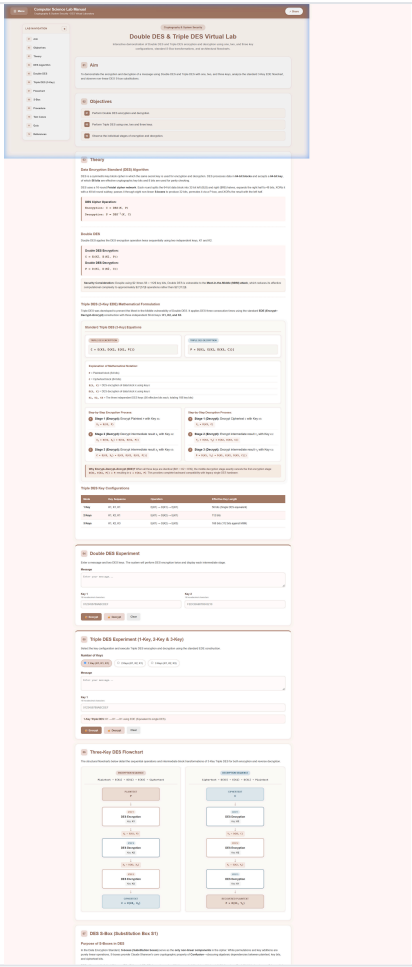


Figure 9: Application running from pulled registry image on `http://localhost:8081` with full functionality intact.

6. Results and Conclusion

6.1 Summary of Results

Stage / Metric	Observed Value / Status	Outcome
Base Image	nginx:alpine (Linux x86_64)	Minimal footprint (~26.3 MB)
Local Container	`des-container` on Port 8080	HTTP 200 OK • All UI & JS features active
Docker Hub Repository	`rohitaraghavan/des-virtual-lab:1.0`	Publicly published and available
Pulled Container	`des-lab-pulled` on Port 8081	HTTP 200 OK • Dual concurrent execution verified
Console / Network Errors	0 Errors	100% clean execution

6.2 Conclusion

Assignment 9 has been successfully completed. The DES Cryptography Virtual Lab was properly Dockerized using an optimized Nginx Alpine configuration. The application was built locally as version 1.0, published to Docker Hub under the username **rohitaraghavan**, and independently pulled and executed on a separate port. Both containers operate concurrently with complete integrity of all cryptographic algorithms, responsive UI, and interactive calculations.